

BACKEND API

Documentacao Tecnica do Backend

Projeto Escola High Tech: API REST em ASP.NET Core para autenticao JWT, autorizacao por perfis, CRUD escolar, persistencia com Entity Framework Core, validacao, Swagger, logs e testes.

Escopo: todo o backend localizado em [Backend_API](#) , incluindo arquitetura, camadas, classes, regras de negocio, diagramas e tecnologias.

Formato: documentacao tecnica em PDF gerada a partir deste HTML.

Projeto: **form_API**

Repositorio: **SPA-Vue / Backend_API**

Data: **22/05/2026**

Documento: **Backend_API/docs/documentacao-tecnica-backend.pdf**

Indice

1. Visao geral do backend
2. Tecnologias usadas e versoes
3. Tipo de arquitetura
4. Funcao de cada camada
5. Comunicacao entre camadas
6. Diagramas de fluxo, classes e entidade-relacional
7. Funcao de cada classe por camada
8. Regras de negocio
9. Autenticacao, autorizacao e JWT
10. Migrations, FluentValidation, Swagger, logs e testes
11. CRUDs e endpoints

1. Visao Geral do Backend

O backend `form_API` e uma API REST para uma aplicacao escolar. Ele centraliza o cadastro de alunos, professores, usuarios e diretoria, alem de cuidar da autenticacao e da autorizacao por perfis.

Dominio

Gestao escolar com alunos, professores, diretoria, usuarios e perfis.

Interface externa

Controllers HTTP documentados via Swagger/OpenAPI.

Persistencia

EF Core com SQL Server em container e SQLite para desenvolvimento/testes.

Seguranca

JWT Bearer, roles, hash de senha e troca obrigatoria da senha padrao.

Qualidade

FluentValidation, testes unitarios e teste contra SQL bruto.

Observabilidade

Logs diarios gravados em `Backend_API/logs`.

2. Tecnologias Usadas e Versoes

Tecnologia	Versao	Uso no backend
.NET / ASP.NET Core	<code>net10.0</code>	Framework principal da API Web.
Microsoft.AspNetCore.Authentication.JwtBearer	<code>10.0.8</code>	Validacao de tokens JWT no pipeline.
Entity Framework Core	<code>10.0.8</code>	ORM, DbContext, migrations e consultas LINQ.
EF Core SQL Server	<code>10.0.8</code>	Provider do banco principal.
EF Core SQLite	<code>10.0.8</code>	Provider usado no ambiente de desenvolvimento e em testes.
FluentValidation.AspNetCore	<code>11.3.1</code>	Validacao automatica de ViewModels.
Swashbuckle.AspNetCore	<code>10.1.7</code>	Swagger/OpenAPI e Swagger UI.
SQL Server	<code>2022-latest</code>	Banco em Docker Compose.
Docker .NET SDK/Runtime	<code>10.0</code>	Build e runtime da imagem da API.
xUnit	<code>2.9.3</code>	Framework de testes unitarios.
Moq	<code>4.20.72</code>	Mocks nos testes de controllers/services.
Microsoft.NET.Test.Sdk	<code>17.14.1</code>	Execucao dos testes .NET.
coverlet.collector	<code>6.0.4</code>	Coleta de cobertura quando habilitada.

3. Tipo de Arquitetura

A arquitetura e uma arquitetura em camadas. Essa escolha separa responsabilidades e deixa o codigo mais simples de testar: a camada de API nao sabe detalhes do banco, os services concentram regras, o DataContext configura o modelo e os ViewModels isolam contratos HTTP das entidades persistidas.

API

Controllers, middlewares, JWT, autorizacao, Swagger e codigos HTTP.

Aplicacao

Services e interfaces que executam regras e orquestram dados.

Infraestrutura

DataContext, Repository, migrations, logging, hashing e acesso ao banco.

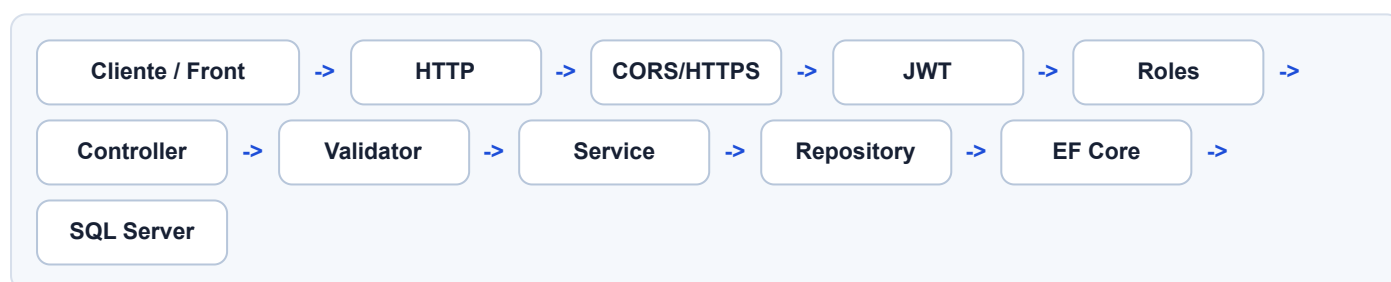
Contratos/Dominio

Models, ViewModels, validators e mapeamentos.

4. Funcao de Cada Camada

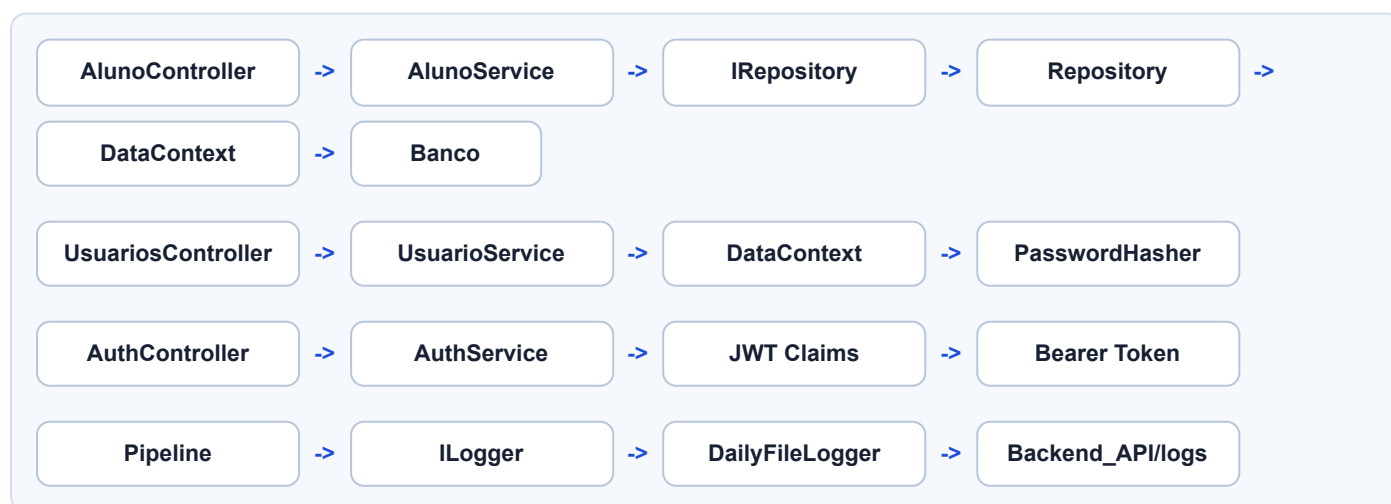
Camada	Pastas	Funcao
Inicializacao	<code>Program.cs</code>	Registra DI, controllers, validators, JWT, policies, EF Core, Swagger, logs, CORS e migrations no startup.
API	<code>Controllers</code>	Recebe HTTP, chama services, aplica atributos de autorizacao e retorna status codes.
Aplicacao	<code>Services</code>	Executa regras de negocio, consulta dados, cria/atualiza/exclui entidades e retorna ViewModels.
Dados	<code>Data</code>	Configura EF Core, relacionamentos, seeds, repository e consultas.
Dominio	<code>Models</code>	Representa entidades persistidas no banco.
Contratos	<code>ViewModels</code>	Define payloads de entrada e saida expostos pela API.
Validacao	<code>Validators</code>	Valida ViewModels antes da execucao das regras de aplicacao.
Seguranca	<code>Security</code>	Hash, verificacao de senha e regra de senha padrao.
Documentacao	<code>Swagger</code>	Personaliza OpenAPI para endpoints protegidos.
Logs	<code>Logging</code>	Provider de arquivo diario para o sistema de logging do ASP.NET Core.

5. Diagrama de Fluxo da Requisicao



O retorno percorre o caminho inverso: banco, EF Core, repository, service, ViewModel, controller e resposta HTTP. O middleware de logs registra início, fim, status e tempo de execucao de cada requisicao.

6. Diagrama de Comunicacao Entre Camadas



7. Fluxo de Autenticacao JWT



O token carrega `sub`, `email`, `nameidentifier`, `name`, `id_perfil` e `role`. A role vem da descricao do perfil e alimenta os atributos `[Authorize(Roles = "...")]`.

8. Diagrama de Classes



Relacionamentos principais: **Perfil 1:N Usuario**, **Usuario 1:N Aluno**, **Usuario 1:N Professor**, **Usuario 1:N Diretoria** e **Professor 1:N Aluno**.

9. Modelo Entidade-Relacional



ALUNO

- Id PK
- Nome
- Sobrenome
- DataNasc
- ProfessorId FK obrigatoria
- IdUsuario FK opcional

DIRETORIA

- Id PK
- Nome
- IdUsuario FK opcional

Relacao	Cardinalidade	Delete behavior
Perfil -> Usuario	1:N	Restrict
Professor -> Aluno	1:N	Cascade
Usuario -> Aluno	1:N opcional	SetNull
Usuario -> Professor	1:N opcional	SetNull
Usuario -> Diretoria	1:N opcional	SetNull

10. Funcao de Cada Classe por Camada

10.1 Inicializacao

Classe/arquivo	Camada	Funcao
Program.cs	Bootstrap/API	Configura DI, controllers, JSON, FluentValidation, CORS, JWT, policies, Swagger, EF Core, migrations automaticas, logs e middleware de requisicoes.

10.2 Controllers

Classe	Funcao	Principais acoes
AlunoController	Endpoint HTTP de alunos.	Listar, buscar por id, buscar por professor, criar, atualizar e excluir aluno.
ProfessorController	Endpoint HTTP de professores.	Listar, buscar por id, criar, atualizar e excluir professor.
DiretoriaController	Endpoint HTTP da diretoria escolar.	Listar, buscar por id, criar, atualizar e excluir integrante da diretoria.
UsuariosController	Endpoint HTTP de usuarios e perfis.	Listar usuarios, buscar por id, listar perfis, criar, atualizar e excluir usuario.
AuthController	Endpoint HTTP de autenticacao.	Login, usuario atual, validacao simples de token, validacao admin e alteracao de senha.

10.3 Services e Interfaces

Classe/interface	Funcao	Observacao
IALunoService	Contrato de operacoes de aluno.	Permite injecao e testes da camada controller.
AlunoService	Regra de aplicacao para aluno.	Usa repository, converte ViewModel para Model e retorna AlunoViewModel.
IProfessorService	Contrato de professor.	Abstrai listagem, busca, criacao, edicao e exclusao.
ProfessorService	Regra de aplicacao para professor.	Orquestra repository e preserva vinculo opcional com usuario.
IDiretoriaService	Contrato de diretoria.	Expõe CRUD da administracao escolar.
DiretoriaService	Regra de aplicacao para diretoria.	Cria, atualiza e remove integrantes da diretoria.
IUsuarioService	Contrato de usuario.	Define CRUD e listagem de perfis.
UsuarioService	Regra de aplicacao para usuarios.	Valida email unico, perfil existente, define senha padrao e usa DataContext direto.
IAuthService	Contrato de autenticacao.	Login, usuario atual e troca de senha.
AuthService	Regra de autenticacao e JWT.	Verifica senha, gera token, monta claims e altera senha do usuario autenticado.

10.4 Data e Persistencia

Classe/interface	Funcao	Detalhes
<code>DataContext</code>	DbContext do EF Core.	Declara DbSet, tabelas, chaves, indices, FKs, delete behavior e seeds.
<code>IRepository</code>	Contrato de persistencia generica e consultas.	Abstrai Add, Update, Delete, SaveChanges e consultas de aluno/professor/diretoria.
<code>Repository</code>	Implementacao de acesso a dados.	Usa LINQ, Include, AsNoTracking, ordenacao e filtros por id/professor.

10.5 Models

Classe	Funcao	Campos relevantes
<code>Aluno</code>	Entidade persistida de aluno.	Nome, Sobrenome, DataNasc, ProfessorId, IdUsuario.
<code>Professor</code>	Entidade persistida de professor.	Nome, IdUsuario, lista de Alunos.
<code>Diretoria</code>	Entidade persistida de pessoa da administracao escolar.	Nome e IdUsuario opcional.
<code>Usuario</code>	Entidade autenticavel.	Nome, Email, Telefone, Senha hash e IdPerfil.
<code>Perfil</code>	Entidade de autorizacao.	Administrador, Contribuinte e Leitor.

10.6 ViewModels e Mapeamentos

Classe	Funcao
<code>AlunoCreateEditViewModel</code>	Payload de criacao/edicao de aluno.
<code>AlunoViewModel</code>	Resposta completa de aluno com professor e usuario.
<code>AlunoSummaryViewModel</code>	Resumo de aluno usado em professor.
<code>ProfessorCreateEditViewModel</code>	Payload de criacao/edicao de professor.
<code>ProfessorViewModel</code>	Resposta de professor com usuario e alunos.
<code>ProfessorSummaryViewModel</code>	Resumo de professor usado em aluno.
<code>DiretoriaCreateEditViewModel</code>	Payload de criacao/edicao da diretoria.
<code>DiretoriaViewModel</code>	Resposta da diretoria com usuario vinculado.
<code>UsuarioCreateViewModel</code>	Payload de criacao/edicao de usuario sem expor senha.
<code>UsuarioSummaryViewModel</code>	Dados publicos de usuario sem senha.
<code>PerfilViewModel</code>	Contrato de saida para perfis.
<code>LoginRequestViewModel</code>	Payload de login.
<code>AuthResponseViewModel</code>	Resposta do login com token, expiracao, usuario e flag de senha padrao.
<code>AlterarSenhaViewModel</code>	Payload para troca de senha.
<code>MappingExtensions</code>	Metodos de conversao entre Models e ViewModels.

10.7 Validators

Classe	Funcao	Regras principais
<code>AlunoCreateEditViewModelValidator</code>	Valida aluno.	Nome, sobrenome, data <code>dd/MM/yyyy</code> , professor positivo e usuario positivo quando informado.
<code>ProfessorCreateEditViewModelValidator</code>	Valida professor.	Nome obrigatorio e usuario positivo quando informado.
<code>DiretoriaCreateEditViewModelValidator</code>	Valida diretoria.	Nome obrigatorio e usuario positivo quando informado.
<code>UsuarioCreateViewModelValidator</code>	Valida usuario.	Nome, email valido, telefone e perfil positivo.
<code>LoginRequestViewModelValidator</code>	Valida login.	Email valido e senha obrigatoria.
<code>AlterarSenhaViewModelValidator</code>	Valida troca de senha.	Senha atual, nova senha forte, confirmacao e bloqueio da senha padrao.

10.8 Security, Swagger e Logging

Classe	Camada	Funcao
<code>PasswordHasher</code>	Security	Gera hash PBKDF2-SHA256 com salt e verifica senha informada.
<code>DefaultPasswordPolicy</code>	Security	Define a senha padrao e detecta se o hash ainda corresponde a ela.
<code>AuthorizeOperationFilter</code>	Swagger	Marca endpoints protegidos no OpenAPI e adiciona respostas 401/403.
<code>FileLoggerOptions</code>	Logging	Configura diretorio, prefixo e nivel minimo de log.
<code>FileLoggerExtensions</code>	Logging	Expõe <code>AddDailyFileLogger</code> para registrar o provider no builder.
<code>DailyFileLogger</code>	Logging	Implementa <code>ILogger</code> e envia mensagens para o provider.
<code>DailyFileLoggerProvider</code>	Logging	Cria arquivo diario e grava linhas de log com data, nivel, categoria, evento, escopo e execucao.

11. Regras de Negocio

- Todo endpoint de negocio exige token JWT valido.
- **Administrador** possui acesso completo: consulta, cadastro, edicao e exclusao.
- **Contribuinte** pode consultar, criar e atualizar alunos, professores e diretoria, mas nao pode excluir.
- **Leitor** pode consultar endpoints autenticados de leitura.
- Somente administrador cria, atualiza e exclui usuarios.
- Somente administrador lista perfis para cadastro de usuario.
- Usuarios criados pela API recebem a senha inicial `Senha@252525`.
- Quando o login detecta uso da senha padrao, a resposta retorna `deveAlterarSenhaPadrao = true`.
- Senha nunca trafega em respostas e nunca e persistida em texto puro.
- Email de usuario e unico; duplicidade gera erro de regra de negocio.
- Perfil informado no cadastro/edicao de usuario precisa existir.
- Aluno precisa apontar para professor valido.
- `IdUsuario` em aluno, professor e diretoria e opcional; quando informado deve ser positivo.
- Data de nascimento de aluno deve usar `dd/MM/yyyy`.
- Ao excluir usuario, vinculos opcionais em aluno, professor e diretoria ficam nulos.
- Ao excluir professor, os alunos vinculados seguem cascade conforme configurado no relacionamento obrigatorio.

12. CRUDs e Endpoints

Entidade	Metodo e rota	Autorizacao	Funcao
Auth	POST <code>/api/Auth/login</code>	Publico	Autentica e retorna JWT.
Auth	GET <code>/api/Auth/me</code>	Autenticado	Retorna usuario atual.
Auth	POST <code>/api/Auth/alterar-senha</code>	Autenticado	Altera senha.
Aluno	GET/POST/PUT/DELETE <code>/api/Aluno</code>	Leitura autenticada; escrita Admin/Contribuinte; delete Admin.	CRUD de alunos.
Professor	GET/POST/PUT/DELETE <code>/api/Professor</code>	Leitura autenticada; escrita Admin/Contribuinte; delete Admin.	CRUD de professores.
Diretoria	GET/POST/PUT/DELETE <code>/api/Diretoria</code>	Leitura autenticada; escrita Admin/Contribuinte; delete Admin.	CRUD de diretoria.
Usuarios	GET/POST/PUT/DELETE <code>/api/usuarios</code>	Consulta Admin/Contribuinte; escrita Admin.	CRUD de usuarios.
Perfis	GET <code>/api/usuarios/perfis</code>	Administrador	Lista perfis disponiveis.

13. Migrations e Banco

Migration	Funcao
<code>20260521160031_InitialSqlServer</code>	Cria estrutura inicial com Professores e Alunos.
<code>20260521194537_SeedFiftyRecords</code>	Insere massa inicial maior de professores e alunos.
<code>20260521202457_AddJwtAuthUsuariosDiretoria</code>	Adiciona JWT no modelo: Perfil, Usuario, Diretoria, FKs IdUsuario, indices, seeds de 3 perfis e 20 usuarios.

No startup, a API aplica migrations automaticamente quando usa SQL Server. Para SQLite, a API usa `EnsureCreated`.

14. Logging

Logs ficam em `Backend_API/logs/backend-api-YYYYMMDD.log`. No Docker Compose, o volume `../Backend_API/logs:/app/logs` grava os logs do container na pasta do projeto.

Origem	Evento
Startup	Ambiente, diretorio de logs e preparacao do banco.
Middleware	Inicio/fim de requisicao com metodo, rota, usuario, status e tempo.
Controllers	Excecoes completas em falhas de CRUD.
Auth	Login recusado, login aceito e troca de senha.

15. Swagger/OpenAPI

Swagger documenta rotas, payloads, status codes e segurancia Bearer. O filtro `AuthorizeOperationFilter` identifica endpoints protegidos e anexa o esquema JWT.

1. Executar `POST /api/Auth/login`.
2. Copiar o token.
3. Clicar em `Authorize`.
4. Informar `Bearer {token}`.

16. Testes Unitarios

Os testes ficam em `Backend_API/form_API.Tests`.

Grupo	O que valida
Controllers	Respostas principais de Aluno e Usuarios.
Services	Criacao, hash de senha, duplicidade de email, login e troca de senha.
Validators	Regras de entrada de aluno, professor, usuario e senha.
Security	Protecao contra uso de SQL bruto.

Comando: `dotnet test form_API.Tests/form_API.Tests.csproj`.

17. Execucao e Deploy Local

Acao	Comando
Rodar API local	<code>cd Backend_API && dotnet run</code>
Build	<code>dotnet build form_API.csproj</code>
Testes	<code>dotnet test form_API.Tests/form_API.Tests.csproj</code>
Docker Compose	<code>docker compose -f docker/docker-compose.yml up --build</code>
Swagger no Compose	<code>http://localhost:5000/swagger</code>

Observacao tecnica: a chave JWT e a senha do SQL Server nao ficam versionadas. Em desenvolvimento, preencha o arquivo local `.env` a partir de `.env.example`. Em ambiente real, use variaveis de ambiente ou secrets do provedor de deploy.